

Mini-Project 3: Predictive Modeling and Optimization for Real Estate Investment

Problem Framing

A real estate investment company needs help making two connected decisions: how much each house is worth, and which houses to renovate under a fixed budget. Pure prediction isn't enough on its own (a price estimate doesn't tell you what to buy), and pure optimization isn't enough either (you can't allocate budget without knowing which properties will pay off).

The pipeline answers both:

1. **Neural network** trained on the Ames Housing dataset (2,930 properties, 82 features) to predict **SalePrice** from a small set of inputs.
2. **Linear program** (a 0/1 knapsack, technically) that uses those predictions to pick a subset of houses that maximizes profit under a fixed renovation budget.

This is a regression problem, not classification, because the target is a continuous dollar amount and the company cares about how much each house is worth, not which bucket it falls into.

Feature Selection Justification

The project allows 6 to 8 input variables. We picked 8:

Feature	Why
Overall Qual	Top-correlated feature in Ames (overall material and finish quality, correlation 0.80 with price)
Gr Liv Area	Above-grade living area, the main size driver
Total Bsmt SF	Basement size, extra usable space
Garage Cars	Garage capacity, a strong amenity signal
Year Built	Newer houses usually sell for more
Year Remod/Add	When the house was last renovated, directly relevant to Part 4
Neighborhood	Location, location, location (one-hot encoded into 28 binary columns)
Full Bath	Bathroom count

We initially picked **Lot Area** instead of **Year Remod/Add**, but swapped it out after checking the data: **Lot Area** is heavily right-skewed (skew 12.82) and only weakly correlated with price (0.27), while **Year Remod/Add** is much better behaved (skew -0.45, correlation 0.53) and ties cleanly into the renovation-budget framing of Part 4. Empirical val and test scores were within noise either way, so the swap was driven by methodology rather than raw accuracy.

Data Preparation and Baseline

Preprocessing: dropped 2 rows with missing values (negligible loss out of 2,930), one-hot encoded **Neighborhood**, standard-scaled the numeric features. The target was also standard-scaled internally so the network trains on a well-behaved range.

Splits: 64% train, 16% val, 20% test (two nested 80/20 splits, fixed seed). The val set is for comparing architectures and tuning hyperparameters so we don't peek at test during selection. Test was kept for the final report.

Metric: RMSE in dollars. Same units as **SalePrice**, and it penalizes big misses (which matter for investment decisions). MAE and R^2 are reported alongside.

Baseline: predict the training mean for every test row. RMSE about \$90K, R^2 about 0.0. Any real model has to beat this.

Model Comparison

We trained four neural network architectures with MLPRegressor (ReLU, Adam, early stopping):

Model	Architecture	Val R^2	Test R^2
Baseline	mean-predictor	-0.00	-0.00
NN-A	(32,)	0.873	0.867
NN-B	(64, 32)	0.876	0.887
NN-C	(128,)	0.863	0.890
NN-D	(64, 32, 16)	0.869	0.875

All four crush the baseline and cluster between R^2 0.86 and 0.89. Diminishing returns kick in fast at this dataset size.

Key takeaways:

- Depth slightly beats width: the wide single-layer net (NN-C) was the worst on val.
- Going past 2 layers didn't help: NN-D's third layer added complexity for no gain.
- Val and test scores match closely for every model, so no overfitting to either split.
- Gaps between top models are under 0.01 R^2 , which is noise-range. Multiple random seeds would firm this up.

Winner (chosen on val): NN-B. Test R^2 0.887, RMSE about \$30K.

We then ran a 4 by 3 = 12-combo hyperparameter sweep on NN-B over **alpha** (L2 weight decay) and **learning_rate_init** (Adam step size). The best combo (**alpha=0.1**, **lr=0.01**) bumped val R^2 to 0.883, but test R^2 stayed flat at 0.884. The grid was mostly flat, confirming the model isn't very sensitive to these knobs. To meaningfully push past R^2 0.89 we'd need more features or a different model class (e.g. gradient boosting), not finer hyperparameter sweeps.

Optimization Setup

The company has limited renovation capacity. The question: which houses should it pick to maximize expected investment gain?

The optimization runs on the **586 test-set properties**, the same houses the neural network never saw during training. We use the tuned NN-B model from Part 3 to score those properties. Restricting to the test set matters: training and validation rows would give the NN artificially perfect predictions, conflating model fit with real out-of-sample signal.

Two quantities are derived per property:

Expected Gain is the NN's residual on out-of-sample data:

$$\text{ExpectedGain} = \text{PredictedPrice} - \text{SalePrice}$$

A positive value flags the property as priced lower than the NN's typical pattern would suggest, which is a candidate worth a closer look. A negative value means the NN thinks the property sold for more than expected. This is a heuristic, not a certified market-mispricing signal: the NN sees the same features the market sees, so the residual reflects either genuine noise the model couldn't capture or its own prediction error.

Renovation Burden is a dimensionless proxy for the effort required to renovate each property, built from three real Ames features and standardized so the components contribute on equal footing:

$$\text{RenovationBurden} = z(10 - \text{OverallCond}) + z(2026 - \text{YearRemodAdd}) + z(\text{GrLivArea})$$

where $z(\cdot)$ z-scores the column (subtract mean, divide by standard deviation). A final shift puts the minimum at 0 so all values are non-negative. The three terms capture condition (worse = more work), age since last remodel (older = more modernization), and size (larger = more labor). Z-scoring removes the unit mismatch between condition (1-10 scale), years (decades), and area (thousands of sqft).

The budget is derived from the data rather than set by hand: the company is assumed to handle 10% of the total renovation workload across the 586 candidates, so $\text{BUDGET} = 0.10 * \text{sum}(\text{RenovationBurden})$.

The LP is a classic 0/1 knapsack:

- **Decision:** for each property, x_i is 0 (skip) or 1 (invest)
- **Objective:** maximize the sum of $\text{ExpectedGain}_i * x_i$ over selected properties
- **Constraint:** sum of $\text{RenovationBurden}_i * x_i$ must stay at or below BUDGET
- **Solver:** PuLP's CBC (open-source MILP solver), checked for Optimal status before results are trusted.

Key Insights

Selected portfolio: 60 of 586 candidates (10.2%) for \$2.38M total Expected Gain (about \$40K per property). The capacity constraint binds at 100%; every available unit of renovation budget is used.

The optimizer ranks by NN residual, not certified mispricing. ExpectedGain measures how far each property deviates from the NN's typical Ames pricing pattern. The model sees the same features the market sees, so residuals carry information but cannot reveal what the market doesn't already price in. Treat the LP output as a candidate list to investigate, not a final buy list.

The burden constraint creates a genuine trade-off. High-gain properties often carry high renovation burden (poor condition, large area, or long-unmodeled age), so the optimizer cannot simply stack the top-gain rows. The top selected pick has the maximum burden in the entire candidate pool (11.34) but is taken anyway because its gain (\$215K) is roughly 5x the next-best gain. Outliers can win when their gain is large enough to justify the burden cost.

The picks tilt newer in remodel age but match the dataset on condition. Average years since remodel is 31 for selected vs 42 dataset-wide; the age term in the burden formula systematically pushes toward more recently modernized homes. Average condition is 5.5/10 in both selected and overall, so condition is not the differentiator. Living area is slightly larger on average for picks (1,700 vs 1,513 sqft), driven partly by one large outlier dragging the mean up.

The budget is dataset-driven, not arbitrary. Setting BUDGET to 10% of total burden means the constraint scales with the composition of the candidate pool. A pool of older, larger, worse-condition homes produces a tighter effective budget than one of newer, smaller, well-maintained ones, which is the right behavior for a capacity-limited firm.

Limitations

- **Single random seed.** Every comparison (NN-A vs NN-B, the tuning grid, the log-transform check) used `random_state=42`. The val R^2 gaps between top models are under 0.01, which is in noise range. Running 3 to 5 seeds would tell us whether NN-B truly beats NN-A or just got lucky on this split.
- **ExpectedGain conflates prediction error with mispricing.** The quantity PredictedPrice - SalePrice captures both genuine undervaluation and NN prediction error. A property with a large positive gain might be a real bargain, or the NN might simply be wrong about it. With ~\$30K test RMSE, the uncertainty on individual gain estimates is substantial and isn't quantified.
- **Renovation Burden is a proxy without dollar grounding.** The burden formula (condition + age + size terms) captures the right intuitions but has no connection to actual construction cost. The 10% budget fraction is similarly a calibration choice. Sensitivity to these design decisions isn't tested in the current implementation.
- **ExpectedGain can be negative, and the LP handles this implicitly.** Properties where the NN predicts less than the sale price contribute negative gain if selected. The LP avoids these automatically when optimizing, but there's no explicit filter, so near-zero or slightly negative properties near the budget boundary could be selected in edge cases.
- **Hyperparameter search was narrow.** We tuned `alpha` and `learning_rate_init` but didn't touch `solver`, `activation`, or `batch_size`. Lbfgs in particular often outperforms Adam on small datasets like this one.

Conclusion

The pipeline answers the original business question end to end. The neural network turns 8 features into a roughly \$30K-RMSE price estimate (test R^2 about 0.89), well above the mean-predictor baseline. The linear program then uses those predictions to identify undervalued properties and selects the optimal portfolio under a renovation capacity constraint.

The prediction step is robust: NN-B beat 3 alternative architectures and held up across 12 hyperparameter combos, all chosen on a held-out val set. The optimization step is mathematically correct (LP returns Optimal status, constraint honored), but its interpretation rests on two design choices: that ExpectedGain =

PredictedPrice - SalePrice is a meaningful proxy for undervaluation, and that the RenovationBurden formula captures the right property characteristics. Both are defensible but neither is ground truth. The honest takeaway: prediction tells us what each house is worth relative to its features; optimization tells us which undervalued houses we can actually afford to take on given capacity limits. Tightening the burden formula with real construction-cost data is the main thing a real engagement would need.